Technische Hochschule Deggendorf

MDM-M-2 Case Study Intelligent Systems in Production

Binary Classification of AOI Alarms into True Defects and False Calls

Second check of AOI results with supervised machine learning
(Siemens production data)

Bibliography Report

Team 1

Yu Han
Melisa Subasi
Meron Hagos
Alejandro Rivas

Under the supervision of:
Prof. Dr.-Ing. Hamidreza Heidari

Date of submission: 03.11.2026

Abstract

In electronics assembly, automated optical inspection (AOI) checks every solder joint after soldering and sends every flagged board to a manual inspection station. Most of these alarms are false calls: in public Siemens production data, about 99 % of the flagged solder joints are false calls. A second check with supervised machine learning after the AOI could release many false calls without manual inspection, and Siemens is working on this problem in its Erlangen electronics factory today.

This report reviews 18 sources, from the general to the specific. Earlier studies show that classifiers such as random forests can recognise false calls from AOI measurements. On the same Siemens data, however, models that meet the targets on a test set from the same period (at most 1 % missed defects, at least 40 % fewer manual checks) miss 5–15 % of the true defects on every later production period. The literature on dataset shift and concept drift, cases from manufacturing, and the authors of the data and of the study themselves point to drift, a change of the data over time, as the cause; an unstable method for setting the decision threshold adds to it. The standard remedy in the drift literature is to retrain the model regularly on recent data. In related applications, a binary fraud classifier with less than 1 % positives and a quality model on a semiconductor materials line, regular retraining reduced the loss in performance. For the second check of AOI results, this remedy has not been tested in the literature we reviewed.

Our project addresses this gap. Its methodology is a supervised binary classifier, the random forest and balanced random forest of Pfab and Rothering, reimplemented with our own code as the baseline, combined with two adaptation strategies from the literature: decision threshold selection (Neyman–Pearson classification, prior probability shift adjustment) and concept drift adaptation by retraining (periodic and drift-triggered). A design of experiments varies the threshold method, the retraining schedule, the training window and the model. The objective is that on every later production period at most 1 % of the true defects are missed, as a hard quality limit, and at least 40 % of the false calls are released; the expected saving lies between 40 % and the 80–90 % that the second check reaches on the test set.

Contents

Abstract i

1 Introduction 1

1.1 Motivation . . . 1

1.2 Key terms . . . 3

2 Materials and Methods 4

2.1 Reviewed literature . . . 4

2.2 Data: the input of the second check . . . 9

2.3 Methodology . . . 10

2.3.1 The classifier: random forest and balanced random forest . . . 12

2.3.2 The two adaptation strategies . . . 12

2.4 Output: objective and targets . . . 15

2.5 Factors and limitations . . . 15

2.6 Summary of the reviewed works . . . 17

3 Discussion 18

3.1 Why the earlier results look better than they are . . . 18

3.2 Limitations of the reviewed sources . . . 18

4 Conclusion 18

References 18

List of Figures

1 The reviewed literature from general to specific. Each box is one level of the review, with its sources on the right; Section 2.1 follows them from top to bottom. The line below the funnel is our project. . . . 1

2 The groups of Table 1 as points: missed true defects (horizontal, further right is worse) and false calls released (vertical, higher is better). B: test set with an ideal threshold; C: test set with the threshold of [7], zoomed in the inset; D: the five later periods of [7]; E: our baseline, mean of the later periods. Dashed lines: the limits of F, at most 1 % missed and at least 40 % released; the 40 % line is also the target A of Siemens. Green: the target zone. . . . 2

3 Share of flagged boards with a true defect per production week (bars, left axis; red: weeks above 3 %) and number of flagged boards per week (line, right axis). Data of [4], our analysis. . . . 6

4 Missed true defects of the two models of [7] on the test set and on the five later periods (slices), mean and standard deviation of their runs [7, p. 7, Table IV; p. 11, Table V]. Dashed line: the target of at most 1 %. 7

5 Minimum number of true defects in the left-out data so that the threshold keeps the share of missed true defects below α with the stated confidence, $n \geq \log \delta / \log(1 - \alpha)$ [14]; dark bars 95 %, light bars 90 % confidence; computed by us. 8

6 Split of the data in time order, as in [7]. Top: the first half (days 0–84) is used for training and for a test set from the same period; the second half is cut into five later periods P1–P5 with about 44,000 joints each. Bottom: share of true defects among the flagged joints in each part. Our analysis of the data of [4]. 10

7 Our system for one later period k. The classifier f_k scores each flagged joint, and the decision rule compares the score $s(x)$ with the threshold t_k: joints at or above it go to the manual station, joints below it are released. Strategy 1 (decision threshold selection) sets t_k; Strategy 2 (concept drift adaptation) retrains f_k on the labelled joints of earlier periods. The output are the two responses FNR_k and TNR_k. In the baseline of [7], both strategies are switched off: t and f are set once. 11

8 Technical route of our project: from the problem on later periods (Table 1) to its two causes, the methods from the literature and the factors of our test plan. Numbers in brackets are the sources. 13

9 Parameter diagram (P-diagram) of our system: input, system and output (responses), with the control factors that our test plan varies (top) and the noise factors that act on the system but are not controlled (bottom). 16

List of Tables

1 What a second check after AOI has reached so far. About 99 % of the flagged joints are false calls, so the share of false calls released is roughly the share of manual checks saved. 2

2 Key terms used in this report. 3

3 The 18 reviewed sources in the order of Figure 1: the finding we use and where it is in the source. 5

4 Structure of one row of the Siemens AOI data [4] and its use in our project. Counts from our analysis of the data file. 9

5 Model families tested by [7] on the test set, mean of ten runs: false calls released at the 1 % limit with an ideal threshold [7, p. 6, Table III], and missed true defects and false calls released with the threshold set by their method [7, p. 11, Table V]. 12

6 Methods from the literature that we use, the cause each one addresses, the evidence that it can help, and its place in our test plan. 14

7 Studies on AOI false calls compared with our project. 17

8 Studies on drift and retraining compared with our project. 17

1 Introduction

Zero-defect manufacturing relies on detecting defects before a product leaves the factory [1]. In electronics assembly, components are soldered onto printed circuit boards by surface-mount technology (SMT), and AOI is placed after soldering [2]. Every board that the AOI flags goes to a manual inspection station, where an operator decides whether it is a true defect or a false call [4]. A classifier after the AOI, a second check, could release the false calls without a manual check. This review covers 18 sources from 2001 to 2026 on this second check; image-based defect detection is outside its scope, because the second check starts after the AOI has made its measurements. Figure 1 shows the 18 sources from the general goal to our project; Section 2.1 follows it from top to bottom.

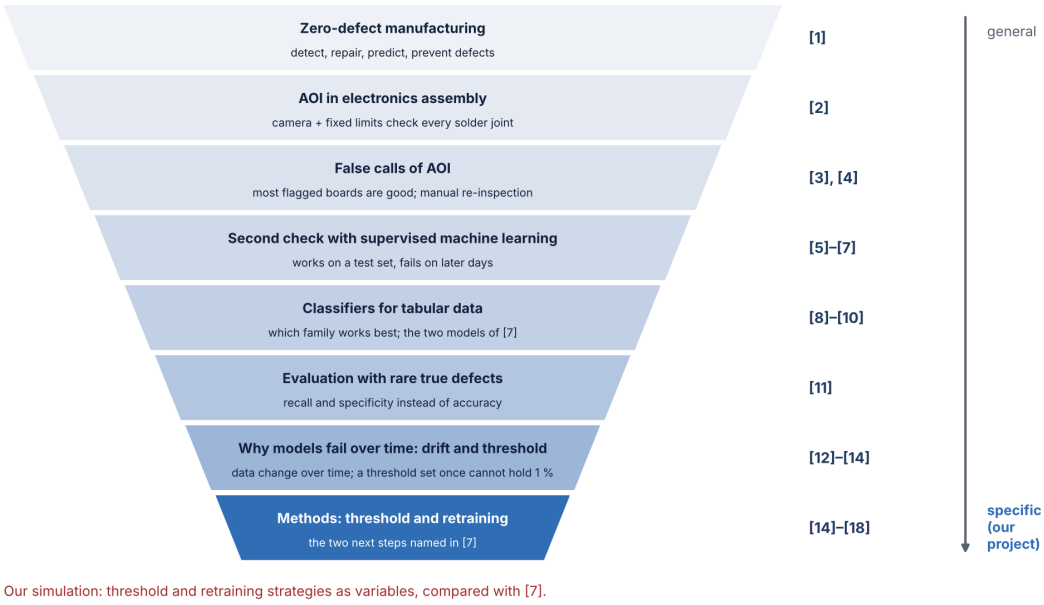


Figure 1: The reviewed literature from general to specific. Each box is one level of the review, with its sources on the right; Section 2.1 follows them from top to bottom. The line below the funnel is our project.

1.1 Motivation

Every board that the AOI flags is checked by a person, and about 99 % of the flagged solder joints are false calls [4, 7]. This costs operator time on every line, and it is a current problem: Siemens is working on an AI second check in its Erlangen factory today, with the target of 40 % fewer false calls [3]. Earlier studies show that a classifier on the AOI measurements can recognise many false calls [5, 6]. The only study that tests such a classifier on later production periods, however, finds that it fails there: it misses 5–15 % of the true defects instead of at most 1 % [7]. A factory cannot use a second check that lets defects through. Table 1 shows what has been reached so far and what is at stake.

Table 1: What a second check after AOI has reached so far. About 99 % of the flagged joints are false calls, so the share of false calls released is roughly the share of manual checks saved.

Group	Source	Setting	False calls released (manual checks saved)	Missed true defects
A	Siemens [3]	industrial target, Erlangen	40 % (target)	–
B	Pfab & Rothering [7, p. 6, Table III]	test set, ideal threshold (best possible)	92.5 % (XGBoost), 90.1 % (RF), 79.6 % (BRF)	1 % (by construction)
C	Pfab & Rothering [7, p. 11, Table V]	test set, threshold as set by their method	79.7 % (BRF), 89.7 % (RF)	0.8 % (BRF), 1.0 % (RF)
D	Pfab & Rothering [7, p. 7, Table IV]	five later production periods	41.5–78.0 % (BRF), 39.7–76.0 % (RF)	5.3–9.4 % (BRF), 10.2–15.0 % (RF)
E	Our baseline	five later periods, 10 seeds, mean	57.9 % (BRF), 55.8 % (RF)	7.5 % (BRF), 9.1 % (RF)
F	Target of [7]	any period	≥ 40 %	≤ 1 %

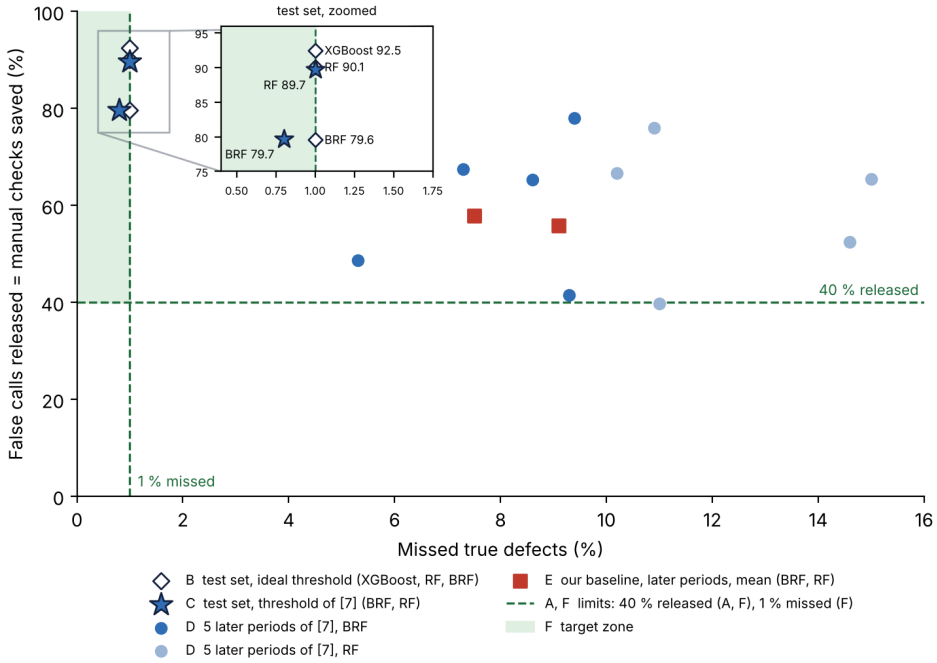


Figure 2: The groups of Table 1 as points: missed true defects (horizontal, further right is worse) and false calls released (vertical, higher is better). B: test set with an ideal threshold; C: test set with the threshold of [7], zoomed in the inset; D: the five later periods of [7]; E: our baseline, mean of the later periods. Dashed lines: the limits of F, at most 1 % missed and at least 40 % released; the 40 % line is also the target A of Siemens. Green: the target zone.

Table 1 and Figure 2 compare all results in the two numbers that matter on the line. The vertical axis is the share of false calls released: each released false call is one manual check saved. The horizontal axis is the share of true defects missed: each missed true defect is a defective joint that passes without a manual check. A second check is acceptable only if it meets both limits of [7] (F): at most 1 % missed and at least 40 % released. The 40 % is also the target of Siemens (A) [3].

Saving at least 40 % of the manual checks is not the problem. All results but one lie above the

40 % line. On the test set, the second check releases 79.6–92.5 % of the false calls (B, C). On the five later periods it still releases 39.7–78.0 % (D); the lowest value, 39.7 %, lies just below it. Our baseline releases 55.8–57.9 % (E). In the Siemens data, with 440,274 flagged joints of which 4,622 are true defects, releasing 90 % of the false calls would mean about 400,000 fewer manual joint checks over 133 days.

Keeping the missed true defects at 1 % is the problem. Only the five results on the test set, which comes from the same period as the training data, meet the 1 % limit (B, C). All ten results on the later periods miss 5.3–15.0 % of the true defects (D), and our baseline misses 7.5–9.1 % (E). They all lie to the right of the 1 % line. Applied to the 4,622 true defects of the data set, 5–15 % would mean about 250 to 690 defective joints passing without a manual check, instead of about 46 at 1 %. These are real defects that would reach the customer.

What our project does. Our project tries to move the later periods (D, E) into the green zone: to bring the missed true defects on later production periods back to at most 1 %, while still releasing at least 40 % of the false calls. There is room for this. Missing fewer true defects requires a stricter threshold, and a stricter threshold releases fewer false calls; but on most later periods the released share is 50–78 %, well above 40 %, so part of it can be given up for fewer missed defects. We test the two methods that [7] names as next steps but does not test. A better method for setting the threshold decides how much of this room is used and keeps the 1 % limit with a stated confidence. Retraining on new data lets the model follow the change in the data, so that the points move back to the left. If one or both methods succeed, the second check saves at least 40 % of the manual checks without letting more true defects pass than the 1 % limit allows.

1.2 Key terms

Table 2 explains the terms used in this report. They follow the core study [7], so that our results can be compared with it.

Table 2: Key terms used in this report.

Term	Meaning
Automated optical inspection (AOI)	A camera system after soldering that measures every solder joint and compares each value with fixed limits [2].
Flagged joint, flagged board	A joint with at least one value outside its limits; a board with at least one flagged joint. Every flagged board goes to a manual inspection station [4].
True defect, false call	The label set at the manual station: the joint is really defective, or the alarm was false. True defect is the positive class.
Second check	A classifier between the AOI and the manual station that decides for each flagged joint whether it can skip the manual check.
Missed true defects (slip)	False negative rate: share of true defects that the second check releases as false calls, $1 - \text{recall}$. Target in [7]: at most 1 %.
False calls released (volume reduction)	True negative rate (specificity): share of false calls that skip the manual station. Target in [7]: at least 40 %.
Threshold	The score above which a joint is treated as a true defect; it is set so that recall reaches 99 % on known data.
Drift	A change of the data over time, so that a model trained on old data no longer fits new data [12].
Retraining	Training the model again on newer labelled data during production.

2 Materials and Methods

2.1 Reviewed literature

The review below follows Figure 1 from top to bottom as one argument. Table 3 lists the sources in the same order, with the finding we use and where it is in the source; page numbers refer to the version linked in the reference list.

Table 3: The 18 reviewed sources in the order of Figure 1: the finding we use and where it is in the source.

Level	Source	Type, year	Finding we use	Where in the source
1	[1]	journal review, 2020	280 articles (1987–2018); four strategies: detection, repair, prediction, prevention	abstract
2	[2]	journal review, 2020	how AOI works: cameras, lighting, feature extraction, fixed limits	abstract
3	[3]	press release, 2026	second check at Erlangen; targets 40 % fewer false calls, 85 % less engineering effort	web page
3	[4]	data article, 2024	132 days, one SMT line; labels set by people; "may contain distribution drifts of unknown type"	abstract; Limitations
4	[5]	conference paper, 2020	kNN, neural network, random forest after AOI; threshold for no extra slip	abstract; summarised in [7, p. 2]
4	[6]	conference paper, 2026	XGBoost best balance of recall and precision on Siemens data; LIME, SHAP	abstract
4	[7]	preprint, 2025	split in time order	p. 3, Sec. IV
			best possible at 1 % missed: 92.5 % (XGBoost), 90.1 % (RF) released	p. 6, Table III
			test set: 0.8 % (BRF) and 1.0 % (RF) missed; 79.7 % and 89.7 % false calls released	p. 11, Table V
			later periods: 5.3–9.4 % (BRF) and 10.2–15.0 % (RF) missed	p. 7, Table IV
			data clusters that "did just appear in a certain period"	p. 3, Fig. 2
			next steps: threshold and distribution drifts; "would fail from the first moment"	pp. 7–8, Sec. VII
5	[8]	conference paper, 2022	45 data sets: tree models remain state of the art on tabular data	p. 1, abstract
5	[9]	journal article, 2001	random forest: trees on random samples and features vote	abstract
5	[10]	technical report, 2004	balanced and weighted random forest for rare classes	pp. 2–3
6	[11]	journal review, 2009	accuracy misleading for rare classes; use recall, specificity	abstract
7	[12]	journal survey, 2014	drift = change of $p(X, y)$; retraining on a sliding window as baseline	definition and adaptation sections
7	[13]	journal article, 2025	controlled drift lowers all metrics of a weld-quality classifier	abstract
7, 8	[14]	journal article, 2018	empirical threshold does not hold an error bound; order statistic, $n \geq \log \delta / \log(1-\alpha)$	p. 2; p. 7 (arXiv copy)
8	[15]	journal article, 2002	re-estimate class shares from unlabelled data and adjust outputs	abstract
8	[16]	conference paper, 2004	drift detection: warning at 2, drift at 3 standard deviations	p. 4, Sec. 3
8	[17]	journal article, 2018	fraud, <1 % positives, 75 million transactions; daily update; recent data better	pp. 1–2; pp. 13–14
8	[18]	preprint, 2026	retraining every 5 vs. 100 batches: about 80 % fewer out-of-range predictions; frequency matters most	p. 9; p. 13

The aim of manufacturing quality is that no defective product leaves the factory. Of the four strategies that research on zero-defect manufacturing distinguishes, detection, repair, prediction and prevention, detection is the one that every line uses today [1]. Detection, however, has a cost: every inspection step adds time and work. A second check after an inspection is one way to lower this cost without letting more defects pass, and this is the idea behind our project.

In electronics assembly, defects are detected after soldering by automated optical inspection. Cameras and lighting record each board, and the AOI software measures every solder joint and compares each value with fixed limits [2]. AOI replaced human visual inspection, which is slow and tires, but it moved the problem rather than solving it. Fixed limits are set conservatively so that no defect slips through, and as a result the machine raises many alarms on joints that are in fact good. Every flagged board still goes to a person.

How large this problem is can be seen in data that Siemens published from one of its own lines. Pfab et al. released 132 days of AOI data from an SMT line of Siemens AG in Germany, with the AOI measurements of every flagged joint and the label from the manual inspection station [4]. About 99 % of the flagged joints are false calls. The authors also warn that the labels are set by people and may be wrong, and that "the data may contain distribution drifts of unknown type". Our own first look at the data confirms the drift (Figure 3): the share of flagged boards with a true defect stays between 0.8 and 2.3 % for 15 weeks and then rises to 3.2–6.3 % in the last four weeks, while the number of flagged boards in these weeks is not unusual. What changes is the mix of true defects and false calls, not the volume, and this will matter later. The problem is also current: in October 2026, Siemens and NVIDIA announced a second inspection step at the Erlangen electronics factory that decides for each AOI alarm whether it is a real defect, with the target of 40 % fewer false calls [3]. This is a target, not a result, but it shows that industry is working on exactly this step.

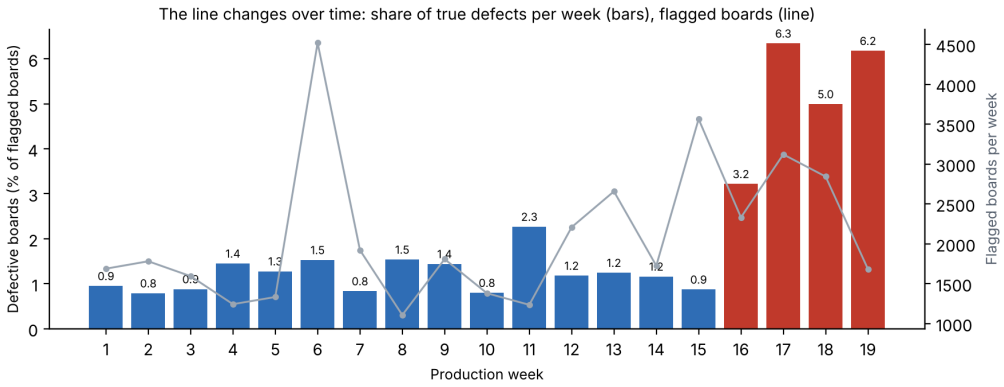


Figure 3: Share of flagged boards with a true defect per production week (bars, left axis; red: weeks above 3 %) and number of flagged boards per week (line, right axis). Data of [4], our analysis.

The natural answer is a second check with supervised machine learning: a classifier that learns from the labels of the manual station and decides for each flagged joint whether it can skip the manual check. Thielen et al. showed that this works in principle on AOI measurements: random forests and neural networks recognised many false calls on large data sets, and the decision threshold could be set so that no additional defects were missed [5]. Albayari et al. reached a similar result on AOI data from Siemens AG, with XGBoost giving the best balance of recall and precision [6]. Neither study reports how many false calls its second check would release at a fixed rate of missed defects, so their results cannot be compared in the terms a factory uses. Both also test the classifier on data from the same period as the training data, by a random split. A factory uses the classifier on the days after training, and neither study tells us what happens then.

Pfab and Rothering answered this question on the public Siemens data, and their study is the core paper of our project [7]. They kept the time order: the first half of the data is used for training and for a test set from the same period, and the second half is cut into five later production periods. Their targets are at most 1 % missed true defects and at least 40 % of the false calls released. On the test set, the best results reported so far for a second check after AOI are theirs. If the threshold is chosen ideally, XGBoost would release 92.5 % of the false calls at the 1 % limit, the random forest 90.1 % and the balanced random forest 79.6 % [7, p. 6, Table III]. With the threshold actually set by their method, XGBoost loses most of this

advantage, and only the random forest and the balanced random forest still meet both targets: they release 89.7 and 79.7 % of the false calls and miss 1.0 and 0.8 % of the true defects [7, p. 11, Table V]. The best model on paper is therefore not the best in use, and the difference is the threshold, a point we return to below. On all five later periods, however, the two models miss 5–15 % of the true defects, already in the first one (Figure 4). The best later result still releases 78 % of the false calls, but misses 9.4 % of the true defects [7, p. 7, Table IV]. The authors conclude that such a model "would fail from the first moment in production". This result changes the question. Whether a second check can recognise false calls is answered; the open question is why it stops working over time, and what can be done about it. Because the authors publish their code and run logs, their result can also be checked and built on directly.

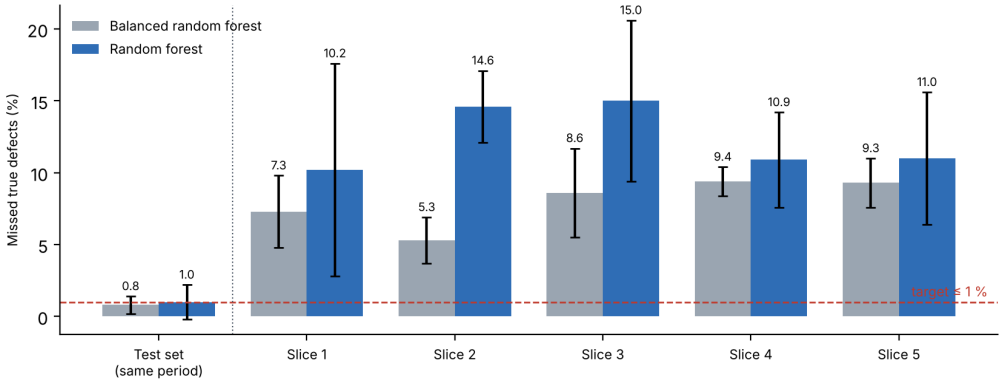


Figure 4: Missed true defects of the two models of [7] on the test set and on the five later periods (slices), mean and standard deviation of their runs [7, p. 7, Table IV; p. 11, Table V]. Dashed line: the target of at most 1 %.

Before turning to the causes, two choices in this study need to be justified, because our project keeps them: the type of model and the way it is measured. AOI measurements are tabular data, one row per joint with numbers and a few categories, and on such data tree ensembles remain stronger than deep learning: on 45 tabular benchmarks, random forests and boosted trees stayed state of the art for medium-sized data [8]. The two models of [7] are of this family. The random forest lets many decision trees, each trained on a random sample of rows and features, vote [9]. The balanced random forest draws the same number of cases from each class for every tree [10], which suits data in which true defects are only about 1 % of the flagged joints. With so few true defects, accuracy is also the wrong measure: a model that calls every joint a false call would be right 99 % of the time and still miss every defect [11]. The study therefore measures, for the two classes separately, the missed true defects and the false calls released, and so do we. The model and the measure are therefore not the reason for the failure; the reason must lie in the data over time.

Pfab and Rothering name two causes themselves. The first is drift. They show data clusters that "did just appear in a certain period of time" and state that "various distribution drifts" occur in the data [7]. In the literature on concept drift, drift is a change of the joint distribution of inputs and labels over time, and a model trained on old data is expected to fail when it occurs [12]. That drift alone is enough to lower a quality classifier in manufacturing was shown in a controlled experiment on friction stir welds, where all metrics dropped once tool geometry, material or lighting changed [13]. Together with the warning of the data authors and the weekly shift in Figure 3, drift is the most likely main cause.

The second cause is the threshold. The threshold of [7] is set once, on the training data, so that 99 % of the known true defects are caught. Tong et al. show why this is not enough [14]: when one type of error must stay below a bound, limiting the error on the data at hand "does not satisfy the [...] error control objective", because the resulting classifiers "are still likely to have [...] errors much larger than" the bound. A threshold set once therefore cannot be trusted to hold 1 % even before the data change, and after a drift it has even less reason to.

Both causes have known remedies, and they are exactly the two next steps that Pfab and Rothering name but do not test: “improving the method for setting the threshold and handling the distribution drifts” [7]. For the threshold, Tong et al. also give a fix. Their method takes the threshold from the scores of a left-out sample in such a way that the error stays below the bound α with a chosen confidence $1-\delta$, a threshold with a safety margin. The price is data: the left-out sample must contain at least $n \geq \log \delta / \log(1-\alpha)$ true defects. For our bound of 1 % with 95 % confidence these are 299 true defects (Figure 5), or roughly 30,000 flagged joints at a defect share of 1 %. The cross-validation folds of [7] are large enough for this, but a short recent period may not be. A second fix addresses the changing share of true defects seen in Figure 3: when the share of a class differs from its share in training, the outputs of a classifier no longer fit, and they can be adjusted by re-estimating the new share from new, even unlabelled data [15]. For a second check this means that the threshold should be set again for each period instead of once.

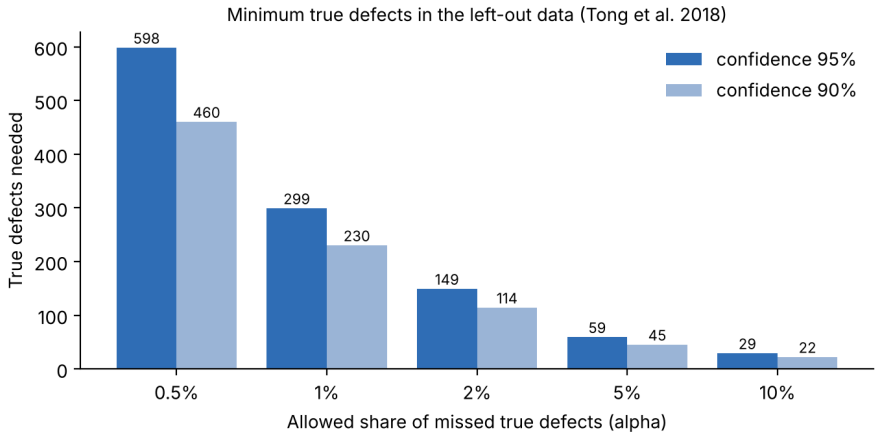


Figure 5: Minimum number of true defects in the left-out data so that the threshold keeps the share of missed true defects below α with the stated confidence, $n \geq \log \delta / \log(1-\alpha)$ [14]; dark bars 95 %, light bars 90 % confidence; computed by us.

For drift, the basic remedy is retraining. In the concept-drift literature, a model that is periodically retrained on a sliding window of the latest data is “often used as a baseline” [12]. The open questions are when to retrain and whether it pays off in practice. One answer to the first question is to retrain only when needed: the drift detection method watches the error rate of the model and retrains when it rises by more than two to three standard deviations above its lowest value [16]. Evidence for the second question comes from two fields close to ours. In credit card fraud detection, a binary task with fewer than 1 % positives on more than 75 million transactions, a classifier updated every day on the newest labelled data performed best, and a model trained on data only a week older was already worse [17]. On a semiconductor materials line, a random forest that predicts product quality degraded after a supplier change and with equipment wear; retraining every 5 batches instead of every 100 gave about 80 % fewer out-of-range predictions, and the frequency of retraining mattered more than the size of the training window [18]. The fraud case is the closest analogue to our task in its statistics, the semiconductor case in its setting; neither is AOI.

This is where the literature stops and our project begins. A second check after AOI is needed and can recognise false calls, but trained once it fails on later production days; drift and a threshold set once explain this, and methods for both exist and have worked elsewhere. For the second check of AOI results, no study has tested them. We therefore reimplemented the method of [7] with our own code as the baseline. Reading their code, we found that the stored threshold belongs to the last model tried during tuning, not to the best model that is finally used; in our reimplementation the threshold is taken from the best model. On this baseline, the threshold method and retraining become the variables of a simulation, and every setting is compared with [7].

2.2 Data: the input of the second check

The input of our second check is the public AOI data set of Siemens [4], the same data that [7] uses. It contains 440,274 flagged solder joints on 39,742 boards from one SMT line, recorded over 132 production days. Each row is one flagged joint. Table 4 shows the columns of a row and how each is used.

Table 4: Structure of one row of the Siemens AOI data [4] and its use in our project. Counts from our analysis of the data file.

Column	Number	Type	Content	Use
`timestamp`	1	date and time	time of the AOI inspection; dates are anonymised; one timestamp per board (39,742)	orders the rows in time; not a model input
`class`	1	0 or 1	label from the manual inspection station: 1 = true defect (4,622 joints, 1.05 %), 0 = false call (435,652 joints)	target y
`inspection_type`	1	category, 5 values	type of AOI inspection applied to the joint	input, one-hot encoded
`meta_feat1–4`	4	categories with 75, 5, 2 and 49 values	anonymised context fields of the joint; their meaning is not published	input, one-hot encoded
`inspection_feat`	70	numbers scaled to 0–1	anonymised AOI measurements of the joint; a mapping file of the data set lists which of them belong to each inspection type (21 to 65)	input x

The inputs of the classifier are therefore the 70 AOI measurements and the five categories, which are one-hot encoded; the timestamp is used only to order the rows. Because the measurements are anonymised and scaled, a model can tell which measurement drives a decision, but not what this measurement means physically. The five inspection types differ strongly: their share of true defects ranges from 0.33 % (type 0, 97,053 joints) to 2.74 % (type 1, 57,673 joints).

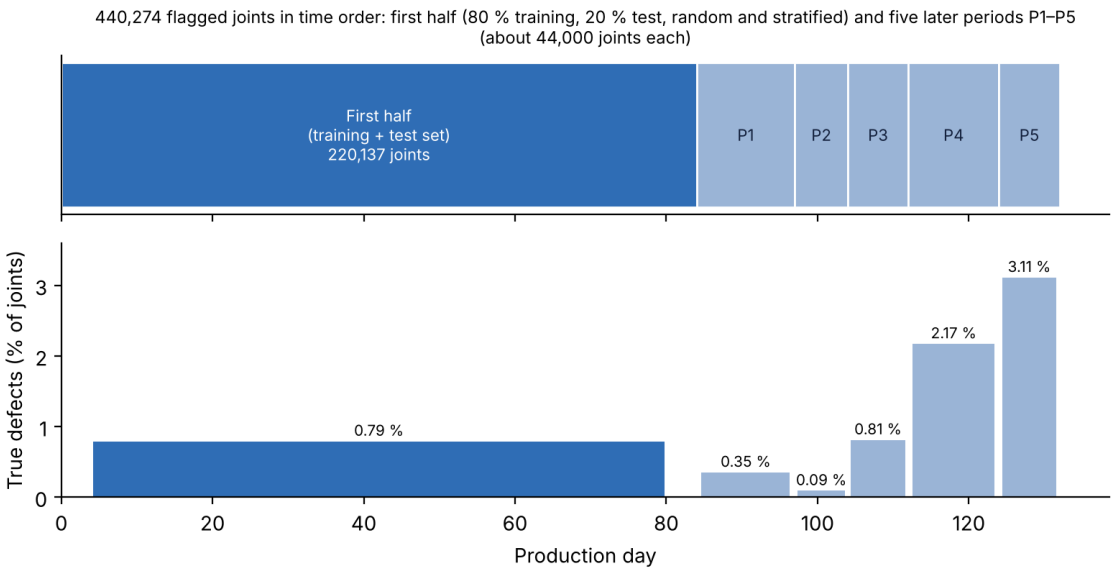


Figure 6: Split of the data in time order, as in [7]. Top: the first half (days 0–84) is used for training and for a test set from the same period; the second half is cut into five later periods P1–P5 with about 44,000 joints each. Bottom: share of true defects among the flagged joints in each part. Our analysis of the data of [4].

Figure 6 shows how the data are used, in the same way as in [7]. The rows are kept in time order. The first half, 220,137 joints from days 0 to 84, is split at random, stratified by the label, into 80 % for training (176,110 joints) and 20 % for the test set (44,027 joints). The classifier is trained on the training part, with the false calls under-sampled at the ratio that [7] logged for each run, and its threshold is set by 5-fold cross-validation on the same part. It is then evaluated on the test set and on the five later periods, which it has never seen. In the methods with retraining, the labelled joints of each period are added to the training data before the next period is classified, either all data seen so far or a recent window.

The bottom of Figure 6 shows why the later periods are hard. In the first half, 0.79 % of the flagged joints are true defects. In the later periods this share drops to 0.35 and 0.09 % and then rises to 0.81, 2.17 and 3.11 %. A model whose threshold was set on a share of 0.79 % meets a share up to four times higher in the last period. This is the change in class share that prior probability shift adjustment (M2) is designed for [15], and part of the drift that retraining should follow [12].

2.3 Methodology

Our methodology is a **supervised binary classifier combined with two adaptation strategies**: **decision threshold selection** and **concept drift adaptation by retraining**. Each term is used in its standard meaning from the cited literature. Figure 7 shows how the parts work together for one later period k; the definitions below describe each part.

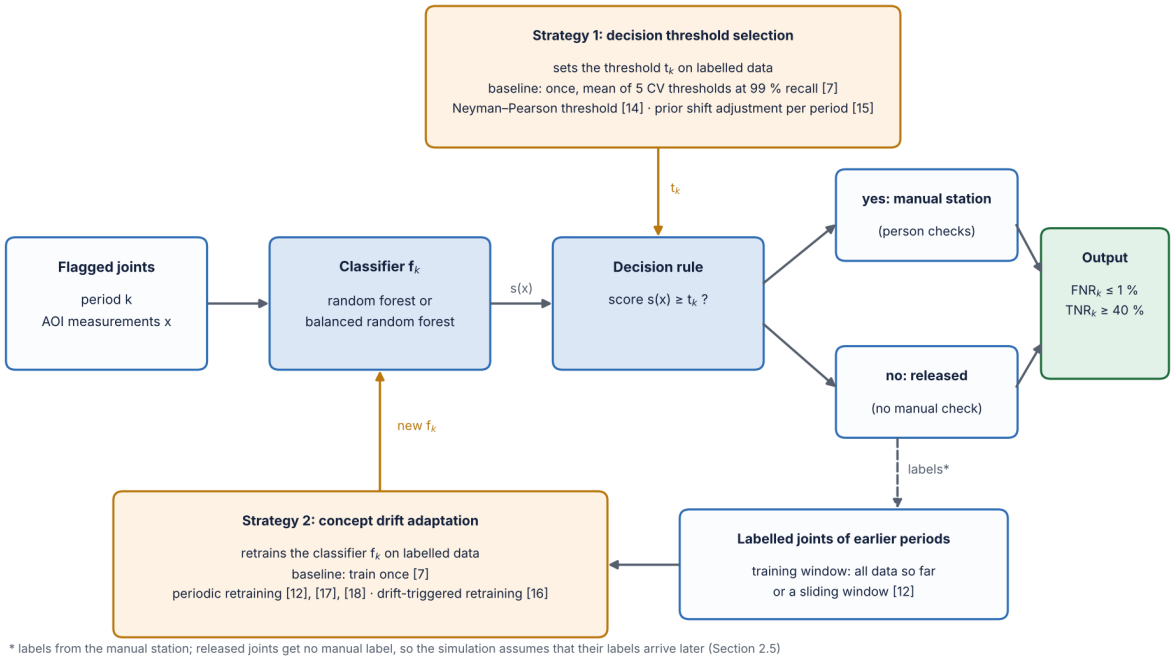


Figure 7: Our system for one later period k. The classifier f_k scores each flagged joint, and the decision rule compares the score $s(x)$ with the threshold t_k: joints at or above it go to the manual station, joints below it are released. Strategy 1 (decision threshold selection) sets t_k; Strategy 2 (concept drift adaptation) retrains f_k on the labelled joints of earlier periods. The output are the two responses FNR_k and TNR_k. In the baseline of [7], both strategies are switched off: t and f are set once.

Task. Each flagged solder joint i has a vector x_i of AOI measurements and a label $y_i \in \{0,1\}$ from the manual inspection station (Section 2.2), with $y_i = 1$ for a true defect (the positive class) and $y_i = 0$ for a false call [4]. The data are ordered in time and split as in [7]: the first half is the training data with a test set from the same period, and the second half forms the later production periods $k = 1,\ldots,5$.

Classifier. A random forest or a balanced random forest [9, 10], trained on labelled joints, gives each joint a score $s(x_i) \in [0,1]$, the estimated probability that it is a true defect (Section 2.3.1).

Decision rule. A joint is sent to the manual station if $s(x_i) \geq t$ and released without a manual check if $s(x_i) < t$, where t is the decision threshold.

Measures. For each period k, the share of missed true defects is the false negative rate, $\mathrm{FNR}_k = 1 - \mathrm{recall}_k$, called slip in [7]; the share of false calls released is the true negative rate, TNR_k, the specificity, called volume reduction in [7]. These class-wise measures replace accuracy, which is misleading when one class is rare [11].

Objective. On every later period k: $\mathrm{FNR}_k \leq 1\,\%$ and $\mathrm{TNR}_k \geq 40\,\%$, the two limits of [7]; Section 2.4 justifies both targets.

Adaptation strategy 1: decision threshold selection. The choice of t. The baseline sets t once, as the mean of the five cross-validation thresholds that reach a recall of 99 % [7]. We compare it with two methods from the literature: Neyman–Pearson classification, which chooses t so that $\mathrm{FNR} \leq \alpha$ holds with a stated probability $1 - \delta$ [14], and prior probability shift adjustment, which re-sets t for each period when the share of true defects changes [15].

Adaptation strategy 2: concept drift adaptation by retraining. The updating of the classifier when the distribution of the data changes over time [12]. The baseline trains once. We compare it with periodic retraining at fixed intervals [12, 17, 18] and with retraining triggered by drift detection [16], each on all

data seen so far or on a recent window [12].

The classifier is held fixed as a control variable, so that every change in FNR_k and TNR_k can be assigned to one of the two adaptation strategies. Section 2.3.1 justifies the classifier, and Section 2.3.2 the methods of the two strategies and their place in our test plan.

2.3.1 The classifier: random forest and balanced random forest

We use the two models of [7], a random forest and a balanced random forest. Table 5 shows the results of all model families that [7] tested on the test set. Two numbers are given for each: the false calls the model could release at the 1 % limit if its threshold were chosen ideally, and the result with the threshold actually set by their method.

Table 5: Model families tested by [7] on the test set, mean of ten runs: false calls released at the 1 % limit with an ideal threshold [7, p. 6, Table III], and missed true defects and false calls released with the threshold set by their method [7, p. 11, Table V].

Model	Released at 1 %, ideal threshold	Missed, actual threshold	Released, actual threshold	Meets both limits?
XGBoost	92.5 %	1.5 % (±2.1)	59.6 % (±42.9)	no: missed above 1 %, very unstable
AutoML	90.6 %	8.3 %	94.9 %	no: missed far above 1 %
Random forest	90.1 %	1.0 %	89.7 %	yes
Balanced random forest	79.6 %	0.8 %	79.7 %	yes
k-nearest neighbours	47.8 %	1.4 %	55.5 %	no: missed above 1 %

Four reasons support the choice of the two random forests. First, they are the only models that meet both limits once the threshold is actually set (Table 5). XGBoost and AutoML could release slightly more false calls with an ideal threshold, but with the threshold set by the method of [7] XGBoost misses 1.5 % of the true defects on average with a very large spread between runs, and AutoML misses 8.3 %. k-nearest neighbours releases far fewer false calls even in the ideal case. Second, the result agrees with the general evidence: on tabular data such as the AOI measurements, tree ensembles remain the strongest model family [8], and random forests are robust to noise [9]. An earlier AOI study also found random forests among the best models on large data sets [5]. Third, the balanced random forest is designed for rare classes [10], and true defects are only about 1 % of the flagged joints; it is the safer of the two, with fewer missed defects at somewhat fewer released false calls. Fourth, a random forest is fast to train, which matters because several of the methods we test retrain the model as often as once a day.

Keeping the models of [7] also makes our results directly comparable with the published ones. Whether a different model would do better is a separate question, which we leave open on purpose: if the model changed at the same time as the threshold or the retraining, an improvement could not be assigned to either.

2.3.2 The two adaptation strategies

The review gives a clear technical route from the problem to the methods we test (Figure 8). The problem is the missed true defects on later periods (groups D and E in Table 1). It has two causes, and the literature offers concrete methods for each of them. Table 6 lists these methods with the source and the place in the source where each one is described, and the factor of our test plan in which it is tested.

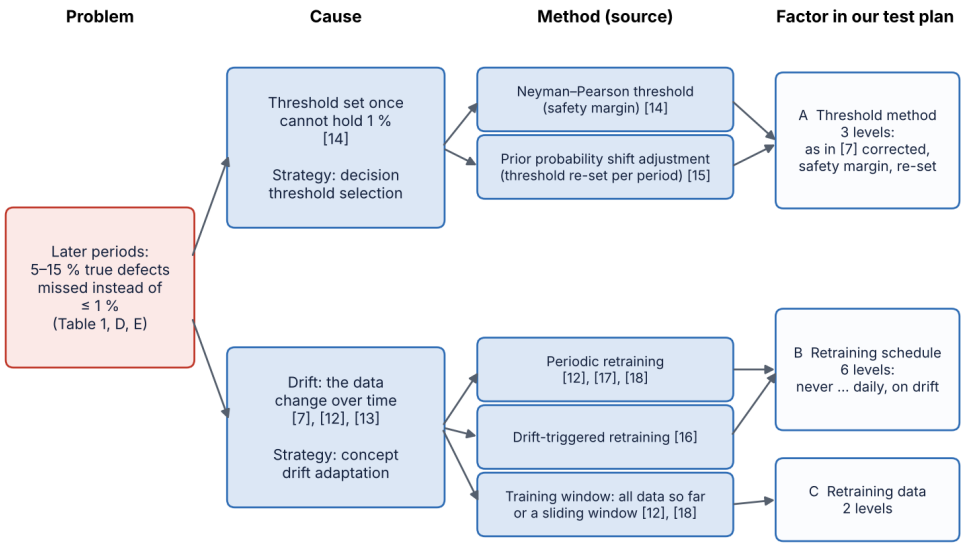


Figure 8: Technical route of our project: from the problem on later periods (Table 1) to its two causes, the methods from the literature and the factors of our test plan. Numbers in brackets are the sources.

Table 6: Methods from the literature that we use, the cause each one addresses, the evidence that it can help, and its place in our test plan.

No.	Method	What it does	Evidence that it can help	Source (where)	Test plan
M0	Baseline: method of [7], threshold corrected	random forest or balanced random forest, trained once; threshold = mean of 5 cross-validation thresholds at 99 % recall, taken from the best model	meets both limits on the test set (Table 1, C)	[7, p. 3, Sec. IV]	A level 1, B level 1
M1	Neyman–Pearson threshold (safety margin)	threshold from the scores of left-out data so that at most $\alpha = 1\,\%$ is missed with confidence $1 - \delta$	the usual threshold does not hold the bound; this one does if at least 299 true defects are left out (Figure 5)	[14, p. 2, p. 7]	A level 2
M2	Prior probability shift adjustment (threshold re-set per period)	re-estimates the share of true defects from new data and adjusts the threshold to it	needed when the share changes, as in the last weeks of the data (Figure 3)	[15], abstract	A level 3
M3	Periodic retraining	retrains the model on the newest labelled data per period, weekly, every 3 days or daily	daily update best in fraud detection with <1 % positives; every 5 instead of 100 batches: about 80 % fewer out-of-range predictions	[12]; [17, pp. 4–5, 13–14]; [18, p. 9]	B levels 2–5
M4	Drift-triggered retraining (drift detection method)	watches the error rate and retrains when it rises 2–3 standard deviations above its lowest value	retrains only when needed; but reacts only after errors have occurred	[16, p. 4, Sec. 3]; [18, p. 13]	B level 6
M5	Training window: all data so far or a sliding window	trains on all labelled data seen so far, or only on the most recent part	a window forgets old data that no longer fit; the frequency mattered more than the window type	[12]; [18, p. 9]	C levels 1–2
M6	Model: random forest or balanced random forest	the two models of [7]	tree ensembles are strongest on tabular data; the balanced variant suits rare true defects	[8, p. 1]; [9]; [10, pp. 2–3]	D levels 1–2

The baseline M0 is the method of [7] as it is, with one correction: the threshold is taken from the best model of the tuning, not from the last model tried. Every other method is compared with it, so that any change in the missed true defects can be traced to one method.

The threshold methods M1 and M2 act on the first cause. They do not change the model; they change where the line between "true defect" and "false call" is drawn. M1 draws it so that the 1 % limit holds with a stated confidence; the price is that the threshold becomes stricter, so fewer false calls are released. Table 1 shows that there is room for this, since most later periods release 50–78 % of the false calls, well above the 40 % limit. M1 also needs enough true defects in the data it is set on, at least 299 for 1 % with 95 % confidence. M2 lets the threshold follow the share of true defects when it changes, as in the last four weeks of the data.

The retraining methods M3 to M5 act on the second cause. They change the model itself, so that it learns

from the newest data and follows the drift. M3 retrains at fixed intervals; the two closest studies found that more frequent retraining helped most [17, 18], which is why our test plan contains four intervals from per period to daily. M4 retrains only when the error rate rises, which saves retraining but reacts only after defects have already been missed; Gao et al. preferred a fixed schedule for this reason [18]. M5 decides which data the model is retrained on. A recent window forgets data that no longer fit, but it also contains fewer true defects, which matters for M1. The methods are therefore not independent, and this is why our test plan combines them in a full design instead of testing each one alone.

2.4 Output: objective and targets

The output of our system are two numbers for each later production period k: the share of missed true defects FNR_k and the share of false calls released TNR_k (Section 2.3). Our objective sets a target for each of them.

> **Objective of our project.** On every later production period $k = 1, \ldots, 5$:
> 1. **Quality (hard limit):** missed true defects $\mathrm{FNR}_k \leq 1\,\%$.
> 2. **Saving (as large as possible, at least 40 %):** false calls released $\mathrm{TNR}_k \geq 40\,\%$.
>
> A setting succeeds only if it meets the quality limit; among the settings that meet it, the one that releases the most false calls is the best.

Why at most 1 % missed true defects. Today, without a second check, every flagged board is inspected by a person, so no flagged defect leaves the station unchecked. Any true defect that the second check releases is therefore a new quality risk, while a false call that still goes to the manual station only keeps the current state [7, p. 4, Sec. V]. Missed defects are therefore much more costly than unnecessary checks. No industry benchmark exists for this limit; the authors of [7], who work at Siemens, set it at 1 % and state that it represents "realistic requirements from the industrial shopfloor" [7, p. 4, Sec. IV]. We adopt it unchanged, because our results are only comparable with theirs under the same limit. We treat it as a hard limit: a setting that misses more than 1 % fails, however many false calls it releases.

Why at least 40 % false calls released. A second check has to pay for itself: it has to save enough manual checks to justify its cost and the remaining risk [7, p. 4, Sec. V]. The 40 % is the minimum saving set in [7] for this reason, and it is also the target that Siemens names for its own second check in Erlangen, "40 % fewer false calls" [3]. A setting that releases less than 40 % therefore does not count as a success, even if it holds the quality limit.

What saving we expect. The 40 % is a minimum, not an expectation. Table 1 gives the range in which the saving can lie. The upper reference is the test set, where the data have not yet changed: there the second check releases 79.7–89.7 % of the false calls within the 1 % limit (group C), and at most 92.5 % with an ideal threshold (group B). No setting can be expected to do better on later periods, where the data have changed. The lower reference is the 40 % limit. Holding 1 % on later periods requires a stricter threshold than in [7], and a stricter threshold releases fewer false calls, so the saving will fall below the 39.7–78.0 % that the later periods reach today with too many missed defects (group D). Our expectation is therefore a saving between 40 % and the test-set value of about 80–90 %, and the position within this range is what our test plan measures. If no setting holds the 1 % limit with at least 40 % released, this is also a result: it shows how far the two adaptation strategies are from making the second check usable, and which of them comes closest.

2.5 Factors and limitations

Figure 9 summarises our system as a parameter diagram (P-diagram), the standard way in the design of experiments to show what goes into a system, what comes out, and which factors act on it. The input

is the AOI data (Section 2.2), the system is the classifier with its two adaptation strategies (Section 2.3), and the output are the two responses of the objective, FNR_k and TNR_k on each later period (Section 2.4).

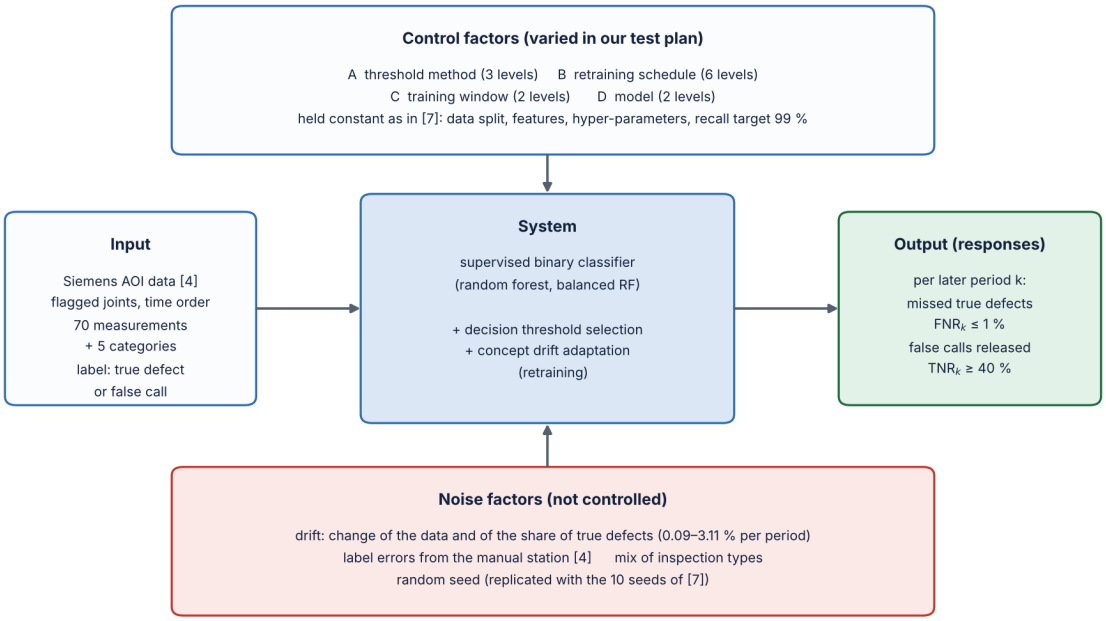


Figure 9: Parameter diagram (P-diagram) of our system: input, system and output (responses), with the control factors that our test plan varies (top) and the noise factors that act on the system but are not controlled (bottom).

Control factors are the factors we set on purpose. They are the four factors of our test plan: the threshold method (A), the retraining schedule (B), the training window (C) and the model (D), with the levels of Table 6. All other settings are held constant at the values of [7]: the data split, the input features, the hyper-parameters logged for each run, and the recall target of 99 % used to set the threshold. Holding them constant makes every result comparable with the baseline and with the published result.

Noise factors act on the system but cannot be set. The most important one is drift: the data and the share of true defects change between periods, from 0.09 to 3.11 % (Figure 6). Our two adaptation strategies are designed to make the system robust against exactly this factor. Further noise factors are label errors from the manual station, whose share is unknown [4], the changing mix of inspection types, whose shares of true defects differ by a factor of eight, and the random seed of the training. The seed is handled by replication: every setting is run with the ten seeds of [7], and results are reported as mean and standard deviation.

Limitations. Seven limitations apply to our project and to the conclusions that can be drawn from it.

1. *One line, one period.* The data come from one SMT line of one company over 132 days [4]. Whether the results hold for other lines, products or longer periods is not known.
2. *Anonymised measurements.* The 70 measurements are anonymised and scaled. A result can say which measurement matters, but not what it means physically.
3. *Label quality.* The labels are set by people and may be wrong [4]. A wrong label counts as an error of the model even if the model was right.
4. *Labels in production.* A joint that the second check releases is not checked by a person and gets no label. Our simulation assumes that every joint receives its label later. In a real line, fewer labels would be available for retraining, so the benefit of retraining may be smaller than in our simulation.
5. *Joint level.* Like [7], we evaluate per solder joint. On the line, a board goes to the manual station if any of its joints is flagged as a true defect, so the saving per board can differ from the saving per joint.

6. *Enough true defects for the threshold.* The Neyman–Pearson threshold needs at least 299 true defects for the 1 % limit with 95 % confidence [14]. A short recent window may contain fewer, and then the method cannot give its guarantee.
7. *Evidence from other fields.* The evidence that retraining helps comes from fraud detection and a semiconductor materials line [17, 18], not from AOI. Whether it transfers to AOI is what our project tests; a negative result is possible and would also be an answer.

2.6 Summary of the reviewed works

Table 7 compares the studies on AOI false calls, and Table 8 compares the studies on drift and retraining. Only [7] evaluates on later production periods, and no study on AOI false calls retrains its model (Table 7). Retraining helped in every study that tested it, but none of these studies is on AOI (Table 8).

Table 7: Studies on AOI false calls compared with our project.

	Data	Classifiers	Evaluation	Training
Thielen et al. [5]	not public	kNN, neural network, random forest	time order not stated	no retraining reported
Albayari et al. [6]	Siemens AG	XGBoost, random forest, DNN, kNN	several train–test splits	no retraining reported
Pfab & Rothering [7]	Siemens, public [4]	kNN, RF, balanced RF, XGBoost, AutoML	test set + 5 later periods	once
Our project	Siemens, public [4]	same as [7] (RF, balanced RF)	same as [7]	**once vs. retrained (6 schedules), 3 threshold methods**

Table 8: Studies on drift and retraining compared with our project.

	Domain	Task	Remedy studied	Main finding
Gama et al. [16]	data streams (benchmarks)	classification	retrain when the error rate rises (DDM)	drift can be detected from the error rate alone
Gama et al. [12]	survey	any	windows, forgetting, retraining	a periodically retrained sliding window is the usual baseline
Dal Pozzolo et al. [17]	credit card fraud	binary, <1 % positives	daily update on the newest labelled data	recent data rank frauds better
Bauer et al. [13]	weld quality (images)	binary	none (effect of drift)	all metrics drop under drift
Gao et al. [18]	semiconductor materials	regression	fixed schedule, window type	frequency matters most
Our project	SMT, AOI (Siemens)	binary, ≈ 1 % positives	threshold method and retraining schedule	tested in P3

3 Discussion

3.1 Why the earlier results look better than they are

Most studies evaluate on a random or stratified split of data from the same period [5,6]. Under drift, such a split mixes past and future and hides the loss in performance [12]. The evaluation in time order of [7] is therefore the one that matches the use in a factory, and it is the reason why we build on this study. The measure matters as well: with about 1 % true defects, accuracy says little, and recall and specificity must be reported for the class that matters [11].

3.2 Limitations of the reviewed sources

Two of the central sources are preprints [7,18]; for [7], the published code and run logs allow us to check the results ourselves. The figures of Siemens [3] are targets, not measured results. The manufacturing evidence for retraining [18] concerns regression, not classification, and the drift experiment of [13] works on images. The limitations of our own project are listed in Section 2.5.

4 Conclusion

The literature leads from the general goal of zero-defect manufacturing [1] to one open question. AOI raises many false calls that people check by hand [2,4], and a second check with machine learning can release many of them [5,6]. On later production days, however, it misses 5–15 % of the true defects instead of at most 1 % [7]. Drift [12,13] and a threshold set once [14,15] explain this failure, and retraining has helped in related fields [17,18], but for AOI no study has tested either remedy.

Position of our project. Pfab and Rothering name as "potential next steps to stabilize the modeling" the "improving [of] the method for setting the threshold and handling the distribution drifts within the dataset" [7], without testing either. Our project tests exactly these two steps. We do not propose a new model; we test whether the standard adaptation strategies, decision threshold selection and concept drift adaptation by retraining, make the second check of AOI results usable in production. The objective is at most 1 % missed true defects and at least 40 % released false calls on every later period, measured against our reimplementation of [7] and against its published result. Whatever the result, it answers a question that the original study left open.

References

[1] F. Psarommatis, G. May, P.-A. Dreyfus, and D. Kiritsis, "Zero defect manufacturing: state-of-the-art review, shortcomings and future directions in research," *Int. J. Production Research*, vol. 58, no. 1, pp. 1–17, 2020, doi: 10.1080/00207543.2019.1605228.

[2] A. A. R. M. Abu Ebayyeh and A. Mousavi, "A review and analysis of automatic optical inspection and quality monitoring methods in electronics industry," *IEEE Access*, vol. 8, pp. 183192–183271, 2020, doi: 10.1109/ACCESS.2020.3029127.

[3] Siemens, "Siemens and NVIDIA advance AI-powered PCB inspection from Digital Twin to factory," press release, Plano, TX, USA, Oct. 21, 2026. [Online]. Available: https://news.siemens.com/en-us/siemens-nvidia-pcba-digital-twin-factory/

[4] K. Pfab, R. Eichler, A. Mallandur, and M. Rothering, "Data of automated optical inspection of surface-mounted technology electronic production," *Data in Brief*, vol. 53, Art. no. 110110, Apr. 2024, doi: 10.1016/j.dib.2024.110110. Data: Mendeley Data, v2, doi: 10.17632/99jzmh9658.2.

[5] N. Thielen, D. Werner, K. Schmidt, R. Seidel, A. Reinhardt, and J. Franke, "A machine learning based approach to detect false calls in SMT manufacturing," in *Proc. 43rd Int. Spring Seminar on Electronics Technology (ISSE)*, Demanovska Valley, Slovakia, 2020, pp. 1–6, doi: 10.1109/ISSE49702.2020.9121044.

[6] J. Albayari, P. Patra, D. Patel, and D. Santos, "Enhancing false call detection in automated optical inspection using XGBoost classifier and XAI algorithms," in *Flexible Automation and Intelligent Manufacturing (FAIM 2025)*, Lecture Notes in Mechanical Engineering. Cham, Switzerland: Springer, 2026, pp. 509–530, doi: 10.1007/978-3-032-05610-8_51.

[7] K. Pfab and M. Rothering, "Towards improved research methodologies for industrial AI: a case study of false call reduction," arXiv:2506.14521 [cs.LG], Jun. 2025. Source code and run logs: `https://gitlab.com/research9841542/FCR_1`. Open-access copy: PDF in our repository.

[8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems 35 (NeurIPS 2022), Datasets and Benchmarks Track*, 2022. Open-access copy: PDF in our repository.

[9] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[10] C. Chen, A. Liaw, and L. Breiman, "Using random forest to learn imbalanced data," Dept. of Statistics, University of California, Berkeley, Tech. Rep. 666, 2004. Open-access copy: PDF in our repository.

[11] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Trans. Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, Sep. 2009, doi: 10.1109/TKDE.2008.239.

[12] J. Gama, I. Žliobaitė, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Computing Surveys*, vol. 46, no. 4, Art. no. 44, pp. 1–37, 2014, doi: 10.1145/2523813.

[13] J. C. Bauer, S. Trattnig, F. Vieltorf, and R. Daub, "Handling data drift in deep learning-based quality monitoring: evaluating calibration methods using the example of friction stir welding," *J. Intelligent Manufacturing*, vol. 37, no. 2, pp. 759–774, 2025, doi: 10.1007/s10845-025-02569-6.

[14] X. Tong, Y. Feng, and J. J. Li, "Neyman-Pearson classification algorithms and NP receiver operating characteristics," *Science Advances*, vol. 4, no. 2, Art. no. eaao1659, 2018, doi: 10.1126/sciadv.aao1659. Open-access copy: PDF in our repository.

[15] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: a simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002, doi: 10.1162/089976602753284446.

[16] J. Gama, P. Medas, G. Castillo, and P. Rodrigues, "Learning with drift detection," in *Advances in Artificial Intelligence – SBIA 2004*, Lecture Notes in Computer Science, vol. 3171. Berlin, Germany: Springer, 2004, pp. 286–295, doi: 10.1007/978-3-540-28645-5_29. Open-access copy: PDF in our repository.

[17] A. Dal Pozzolo, G. Boracchi, O. Caelen, C. Alippi, and G. Bontempi, "Credit card fraud detection: a realistic modeling and a novel learning strategy," *IEEE Trans. Neural Networks and Learning Systems*, vol. 29, no. 8, pp. 3784–3797, 2018, doi: 10.1109/TNNLS.2017.2736643. Open-access copy: PDF in our repository.

[18] M. Gao, J. M. Perathoner, A. L. Bonin, S. Eulig, and G. Klesse, "Robust and reliable AI for predictive quality in semiconductor materials manufacturing with MLOps and uncertainty quantification," arXiv:2605.07752 [cs.LG], 2026. Open-access copy: PDF in our repository.